

Specyfikacja modułu: Access Management

Sekcja aplikacji: Administracja / Admin → Access Management **Wersja:** V1 **Status:** do implementacji (Claude Code) **Data:** 2026-07-22

1. Cel i zakres

Mechanizm sterowania **widocznością sekcji i stron** dashboardu w kontekście wybranego użytkownika.

W V1 istnieje jeden użytkownik — `owner` / „Owner-Admin” (rola `OWNER`) — ale model danych i UI są od początku wieloużytkownikowe, żeby rozszerzenie do V2 nie wymagało migracji plików ani przebudowy ekranu.

Poza zakresem V1 (patrz §11): uwierzytelnianie (login/hasło), poziomy uprawnień read/write/execute, filtrowanie danych per użytkownik, role i grupy dziedziczone.

Ważne rozróżnienie: moduł steruje **warstwą prezentacji**, a nie procesami. Ukrycie strony *Update* nie wyłącza zadań cron ani pobierania danych (§6, W3).

2. Słownik pojęć

Pojęcie	Znaczenie
Zasób (<i>resource</i>)	Sekcja albo strona — najmniejsza jednostka, którą można pokazać/ukryć
Sekcja	Kontener w menu (SPOT, RMB, VAR, FC, Admin)
Strona	Pojedynczy widok (Polish Spot, Spot Daily, Spots History, ...)
Flaga visible	Surowa decyzja użytkownika zapisana w Parquet
Widoczność efektywna	Wynik reguł kaskady (§5, R2) — to, co faktycznie widzi użytkownik
Zasób chroniony (<i>protected</i>)	Zasób, którego nie da się ukryć (Admin, Access Management)
Batch	Jeden zapis (jedno kliknięcie „Zapisz zmiany”) obejmujący N zmian, spięty wspólnym <code>batch_id</code>

3. Rejestr zasobów (*resource registry*)

Rejestr jest **jedynym źródłem prawdy o tym, jakie zasoby istnieją**. Trzymany **w kodzie** (`access/registry.py`), nie w Parquet — zasoby wynikają z routingu aplikacji, a plik Parquet przechowuje wyłącznie decyzje użytkownika. Dodanie nowej strony = jeden wpis w rejestrze, bez migracji danych.

3.1 Struktura wpisu

Pole	Typ	Opis
<code>resource_id</code>	str	Stabilny identyfikator, <code>snake_case</code> , hierarchiczny (<code>spot.polish_spot</code>)
<code>type</code>	enum	<code>section</code> <code>page</code>
<code>parent_id</code>	str null	Dla stron — id sekcji; dla sekcji <code>null</code>
<code>label</code>	str	Etykieta zgodna z menu aplikacji
<code>route</code>	str	Ścieżka / klucz strony w routerze
<code>order</code>	int	Kolejność w menu i w tabeli zarządzania
<code>default_visible</code>	bool	Domyślnie <code>True</code>
<code>protected</code>	bool	Domyślnie <code>False</code> ; <code>True</code> blokuje ukrycie

3.2 Rejestr startowy

Poniższa tabela odwzorowuje rysunek koncepcyjny i obecne moduły dashboardu. **Przy implementacji należy ją zsynchronizować z faktycznym menu aplikacji** — struktura poniżej jest wzorcem, nie listą zamkniętą.

resource_id	typ	parent	label	protected
spot	section	—	SPOT	nie
spot.polish_spot	page	spot	Polish Spot	nie
spot.german_spot	page	spot	German SPOT	nie
spot.spot_daily	page	spot	Spot Daily	nie
spot.spots_history	page	spot	Spots History	nie
spot.update	page	spot	Update	nie
rmb	section	—	Ancillary / RMB	nie
rmb.ancillary_history	page	rmb	Ancillary History	nie
rmb.ancillary_daily	page	rmb	Ancillary Daily	nie
var	section	—	VAR	nie
var.var_kalibracja	page	var	VAR Kalibracja	nie
var.var_raport	page	var	VAR Raport	nie
fc	section	—	Forward Curve	nie
fc.fc_kalibracja	page	fc	FC Kalibracja	nie
fc.fc_raport	page	fc	FC Raport	nie
fc.grafik_dobowo_godzinowy	page	fc	Grafik dobowo-godzinowy	nie
admin	section	—	Admin	TAK
admin.access_management	page	admin	Access Management	TAK

3.3 Zasoby osierocone

Zasób usunięty z rejestru: wiersze historyczne **zostają** w Parquet (log jest niemodyfikowalny), ale są ignorowane przy budowie menu. W zakładce Audyt oznaczane etykietą *orphan*.

4. Model danych — Parquet, append-only

Zgodnie ze standardem pozostałych modułów dashboardu: **log dopisywany, brak UPDATE i DELETE, stan aktywny wyliczany przez filtrowanie najnowszych wierszy. Nie ma osobnego pliku „current”.**

Lokalizacja: katalog danych aplikacji, np. `data/access/`.

4.1 users.parquet

Kolumna	Typ	Opis
ts_utc	timestamp[ms, UTC]	Moment zapisu wiersza
user_id	str	Klucz techniczny, np. owner
login	str	Login (V1 = owner)
display_name	str	Nazwa w UI, V1 = Owner-Admin
role	str	OWNER USER
active	bool	Konto aktywne
note	str null	Komentarz

Stan aktywny użytkownika = wiersz o `max(ts_utc)` dla danego `user_id`. **Seed przy pierwszym uruchomieniu:** owner / Owner-Admin / OWNER / `active=True`.

4.2 access_rights.parquet

Kolumna	Typ	Opis
ts_utc	timestamp[ms, UTC]	Moment zapisu
user_id	str	Kogo dotyczy uprawnienie
resource_id	str	Zasób z rejestru
visible	bool	Decyzja
changed_by	str	user_id operatora (V1 zawsze owner)
source	str	ui seed api
comment	str null	Opcjonalny komentarz z formularza zapisu
batch_id	str (uuid4)	Wspólny dla wszystkich zmian z jednego kliknięcia „Zapisz”

4.3 Wyznaczanie stanu aktywnego

Dla każdej pary (user_id, resource_id) bierzemy wiersz o **maksymalnym** ts_utc; przy równym ts_utc rozstrzyga kolejność wystąpienia w pliku (ostatni wygrywa). **Brak jakiegokolwiek wiersza → wartość default_visible z rejestru.**

4.4 Zapis

Pliki są małe (rzędu setek wierszy), więc: odczyt całości → concat → zapis całości. Zapis **atomowy**: *.tmp + os.replace. Prosty lock file na czas zapisu, żeby UI i ewentualny proces w tle się nie nadpisały.

5. Reguły widoczności

- R1 — domyślność.** Brak wpisu w access_rights.parquet → obowiązuje default_visible z rejestru. Nowo dodana strona pojawia się automatycznie; nie trzeba nic klikać.
- R2 — kaskada.** Strona jest efektywnie widoczna wtedy i tylko wtedy, gdy: visible(strona) = True **AND** visible(sekcja nadrzędna) = True.
- R3 — odwracalność.** Wyłączenie sekcji **nie nadpisuje** flag jej stron. Po ponownym włączeniu sekcji strony wracają do swoich indywidualnych ustawień sprzed ukrycia. *Uzasadnienie: brak destrukcyjnej utraty konfiguracji.*
- R4 — zasoby chronione.** Zasoby protected (Admin, Access Management) są dla roli OWNER zawsze widoczne. Przełącznik w UI jest nieaktywny (disabled) z tooltipem, a **warstwa serwisowa odrzuca** próbę zapisu visible=False — walidacja po stronie logiki, nie tylko UI. Chroni to przed zablokowaniem sobie dostępu do panelu (lock-out).
- R5 — pusta sekcja.** Sekcja, w której żadna strona nie jest efektywnie widoczna, **nie renderuje się w menu** (pusty nagłówek nie ma wartości), a w tabeli Access Management jest oznaczona ikoną ostrzeżenia. (patrz pytanie otwarte O2)
- R6 — natychmiastowość.** Po zapisie cache mapy dostępu jest inwalidowany, a menu przerysowuje się bez restartu aplikacji.

6. Egzekwowanie — dwie warstwy

- W1 — nawigacja.** Pozycje niewidoczne nie są renderowane w menu.
- W2 — strażnik trasy (route guard).** Każde wejście na stronę — również z zakładki, wklejonego URL-a lub przycisku „wstecz” — sprawdza is_visible(user_id, resource_id). Wynik False → widok „**Brak dostępu**” (403) i wpis do logu zdarzeń. Bez W2 ukrywanie w menu jest wyłącznie kosmetyką; obie warstwy są obowiązkowe.
- W3 — procesy w tle.** Zadania cron, aktualizacje danych i zapisy do Parquet **nie podlegają** regułom widoczności. Ukrycie strony Update ukrywa przycisk, ale harmonogram działa dalej.

7. Interfejs użytkownika — strona Access Management

Układ zgodny z rysunkiem koncepcyjnym: drzewo zasobów po lewej, status widoczności po prawej.

7.1 Nagłówek

- Selektor użytkownika** (dropdown) — V1 zawiera jedną pozycję: Owner-Admin (owner). Wybrany użytkownik wyznacza kontekst całej tabeli.
- Badge roli (OWNER).
- Informacja: „Ostatnia zmiana: {ts lokalny} przez {changed_by}”.
- Licznik: „Widoczne: X z Y stron”.

7.2 Tabela zasobów

- Wiersz sekcji** — wyższy, wyróżniony tłem (jak niebieskie belki na rysunku), etykieta wyrównana do lewej, chevron do zwijania/rozwijania.
- Wiersze stron** — wcięcie ok. 24 px, niższe, wizualnie podrzędne wobec sekcji.
- Kolumna „Widoczna”** — przełącznik po prawej stronie wiersza, w jednej kolumnie dla sekcji i stron. **Propozycja: toggle switch** zamiast checkboxa — stan on/off jest czytelny z odległości i lepiej znosi kolorowanie. Checkbox z rysunku pozostaje akceptowalną

alternatywą (pytanie otwarte O3). Kolory: **zielony = widoczna**, szary = ukryta. *Uwaga do standardu UI:* obowiązująca w dashboardie konwencja **czerwony = pozycja short / zielony = long** dotyczy wykresów pozycji. Tutaj stan „ukryta” oznaczamy **szarym**, nie czerwonym, żeby nie mieszać dwóch znaczeń koloru.

- **Stan trójanowy sekcji** — obok przełącznika sekcji indyktor: wszystkie strony widoczne / część / żadna.
- **Wiersze chronione** — przełącznik `disabled`, ikona kłódki, tooltip: „Zasób administracyjny — nie można ukryć”.

7.3 Sterowanie i zapis

- Pole **Filtruj** (po etykiecie i `resource_id`).
- Przyciski: **Zaznacz wszystko**, **Odznacz wszystko** (pomija `protected`), **Przywróć domyślne** (dopisuje wiersze równe `default_visible`).
- Zmiany są **buforowane w stanie sesji**; przycisk **Zapisz zmiany** aktywny wyłącznie gdy istnieją zmiany, obok **Anuluj**.
- Przed zapisem **modal z podsumowaniem różnic**: lista w formie `Spot Daily: widoczna → ukryta`, plus pole na opcjonalny komentarz. Zapis następuje dopiero po „Potwierdź”. Wzorec zgodny z obowiązującą w projekcie zasadą „**system proponuje — użytkownik akceptuje**”.
- Wskaźnik niezapisanych zmian oraz ostrzeżenie przy próbie opuszczenia strony.
- Toast po zapisie: „Zapisano N zmian”.

7.4 Zakładka „Audyt / Historia”

- Tabela z `access_rights.parquet`: `ts` (czas lokalny PL + UTC), użytkownik, zasób (etykieta + id), `visible`, `changed_by`, `comment`, `batch_id`.
- Filtry: użytkownik, zasób, zakres dat. Sortowanie malejąco po `ts`.
- Eksport do **Excela (xlsx)** — spójnie z pozostałymi modułami dashboardu.
- (V1.1, opcjonalnie) Przycisk „**Cofnij ten zapis**” przy `batch_id` — dopisuje nowy batch z wartościami sprzed zmiany; historia nigdy nie jest kasowana.

8. Warstwa serwisowa (API wewnętrzne)

Moduł `access/service.py`:

Funkcja	Zwraca / działanie
<code>list_resources()</code>	Lista definicji z rejestru, posortowana wg <code>order</code>
<code>get_users()</code>	DataFrame aktywnych użytkowników
<code>get_access_map(user_id)</code>	<code>dict[resource_id, bool]</code> — flagi surowe, z uzupełnieniem domyślnych
<code>get_effective_map(user_id)</code>	<code>dict[resource_id, bool]</code> — po zastosowaniu kaskady R2 i R4
<code>is_visible(user_id, resource_id)</code>	<code>bool</code> — używane przez strażnika trasy
<code>save_changes(user_id, changes, changed_by, comment)</code>	Waliduje <code>protected</code> , odrzuca zmiany „bez zmiany”, dopisuje jeden batch, zwraca <code>batch_id</code> , inwaliduje cache
<code>reset_to_default(user_id)</code>	Dopisuje batch z wartościami <code>default_visible</code>
<code>get_history(user_id=None, resource_id=None, date_from=None, date_to=None)</code>	DataFrame do zakładki Audyt

Cache: słownik w pamięci per `user_id`, TTL 60 s **oraz** twarda inwalidacja przy każdym `save_changes` / `reset_to_default`.

9. Inicjalizacja

Przy pierwszym uruchomieniu (brak plików):

1. utworzenie katalogu `data/access/`,
2. `users.parquet` z jednym wierszem `owner` (§4.1),
3. `access_rights.parquet` — **pusty plik z poprawnym schematem**, bez wierszy seed.

Uzasadnienie pustego pliku: reguła R1 pokrywa stan początkowy, a log nie zaczyna się od kilkudziesięciu wierszy technicznych.

Wpływ na istniejące moduły: dodanie filtra w budowniczym menu oraz strażnika trasy w routerze. Logika biznesowa pozostałych modułów pozostaje nietknięta.

10. Testy akceptacyjne (obowiązkowe)

#	Test	Oczekiwany wynik
T1	Pusty <code>access_rights</code>	<code>get_effective_map('owner') = True</code> dla wszystkich zasobów z <code>default_visible=True</code>
T2	Zapis <code>spot.spot_daily=False</code> (ts1), potem <code>=True</code> (ts2 > ts1)	Widoczność efektywna <code>True</code> ; plik zawiera 2 wiersze , nie 1
T3	Trzy kolejne zapisy po 2 zmiany	Liczba wierszy rośnie o 2 / 2 / 2; żaden istniejący wiersz nie zmienia wartości
T4	<code>spot=False</code> , <code>spot.polish_spot=True</code>	<code>is_visible('owner', 'spot.polish_spot') == False</code> ; menu nie zawiera sekcji SPOT ani jej stron
T5	Po T4 ustaw <code>spot=True</code>	<code>spot.polish_spot</code> znów widoczna — bez ponownego klikania stron (reguła R3)
T6	<code>save_changes('owner', {'admin.access_management': False})</code>	Odrzucenie (wyjątek <code>ProtectedResourceError</code>); brak nowego wiersza dla tego zasobu
T7	<code>spot.update=False</code> , wejście bezpośrednio na trasę <code>Update</code>	Widok 403 „Brak dostępu”; treść strony nie renderuje się
T8	<code>spot.update=False</code> , uruchomienie zadania <code>cron</code>	Aktualizacja danych wykonuje się normalnie (reguła W3)
T9	Nowy wpis w rejestrze <code>xyz.new_page</code> , brak wierszy w Parquet	Strona widoczna od razu (reguła R1)
T10	Zapis 3 zmian z komentarzem „test”	W historii 3 wiersze ze wspólnym <code>batch_id</code> i tym samym komentarzem

10.1 Scenariusz kontrolny — obowiązkowy test numeryczny

Stan startowy: sekcja SPOT z 5 stronami, wszystkie widoczne, `access_rights.parquet = 0 wierszy`, licznik „Widoczne: 5 z 5”.

Krok	Akcja	Liczba wierszy w pliku	Menu SPOT	Licznik
1	Odznacz <i>German SPOT</i> i <i>Update</i> → Zapisz	2	3 pozycje	3 z 5
2	Odznacz sekcję <i>SPOT</i> → Zapisz	3	sekcja nie renderuje się	0 z 5
3	Zaznacz sekcję <i>SPOT</i> → Zapisz	4	dokładnie 3 pozycje: Polish Spot, Spot Daily, Spots History	3 z 5

Krok 2 **nie może** dopisać wierszy dla stron podrzędnych — po kroku 2 plik ma dokładnie 3 wiersze. Krok 3 weryfikuje regułę R3.

11. Zakres odłożony (V2)

- Uwierzytelnianie: login, hasło (hash), sesja; podstawienie realnego `user_id` w miejsce stałej `owner`.
- Wielu użytkowników + funkcja „skopiuj ustawienia dostępu od...”.
- Szablony ról: *Analitik*, *Trader*, *Podgląd*.
- Poziomy uprawnień `read` / `write` / `execute` — np. kto może uruchomić *Update*, kto akceptować kalibrację FC i VAR.
- Filtrowanie danych per użytkownik (portfele, obszary rynkowe).
- Wygaszanie dostępu z datą ważności.

12. Pytania otwarte dla implementatora

01. Czy aplikacja ma już warstwę sesji/logowania, z której można pobrać bieżący `user_id`, czy w V1 pozostaje on zaszyty na stałe jako `owner`? (od tego zależy, czy strażnik trasy czyta sesję, czy stałą)
02. Sekcja, w której wszystkie strony są ukryte — ukrywać całą sekcję w menu czy pokazywać pusty nagłówek? (propozycja: ukrywać, reguła R5)
03. Checkbox zgodny z rysunkiem czy toggle switch? (propozycja: toggle)
04. Czy strony akcyjne (*Update*) mają dostać osobne uprawnienie „może uruchomić” już w V1? (propozycja: V2)

13. Pliki do utworzenia i modyfikacji

Plik	Status
access/registry.py	nowy — definicje zasobów (§3)
access/storage.py	nowy — I/O Parquet, zapis atomowy, lock
access/service.py	nowy — logika i cache (§8)
pages/admin_access_management.py	nowy — UI (§7)
budowniczy menu / router	modyfikacja — filtr W1 + strażnik W2 (§6)
tests/test_access.py	nowy — testy T1-T10 + scenariusz §10.1